

Guion corrido de exposición de Campus

Angel Fernando Reyes Moreno · Programación Web II · PA1 | Sección 3677.202620 · Docente Wilder Julio Espinoza Bravo

Duración orientativa: 10 minutos, incluidas las demostraciones. Lee los párrafos como una explicación continua. Las indicaciones entre corchetes no se pronuncian.

Diapositiva 1 Campus | 0:00 a 0:45

[Mantén la cámara encendida. Presenta la portada y pasa a la diapositiva 2.]

Buenas tardes, profesor. Soy Angel Fernando Reyes Moreno, de la sección 3677.202620. Voy a presentar Campus, una aplicación de solicitudes académicas desarrollada para la PA1 de Programación Web II. El objetivo es aplicar los contenidos de las primeras cuatro sesiones mediante un caso que se pueda revisar y probar.

La aplicación permite registrar una solicitud, consultar su detalle y encontrarla mediante filtros. También tiene un directorio que consulta una API externa. Durante la exposición explicaré cómo se organizó el proyecto, qué decisiones técnicas lo sostienen y cómo se comprueba su funcionamiento. Los datos que mostraré son ficticios y corresponden a una demostración educativa.

Diapositiva 2 El caso y el procedimiento | 0:45 a 1:35

[Señala el listado de la captura. Todavía no cambies al navegador.]

El caso consiste en organizar solicitudes como constancias de estudios, certificados de notas, cambios de horario y reservas de matrícula.

El desarrollo se entiende en cuatro pasos: definir los datos con TypeScript; organizar las pantallas y los servicios en Angular; conectar el formulario con sus validaciones y rutas; y consumir una API REST. Después se comprueba el recorrido completo con pruebas.

En la captura se ve el listado principal, con búsqueda y filtros por estado. Los seis registros iniciales permiten explorar la interfaz desde el primer uso. Sus estados son ejemplos. Una solicitud nueva comienza como pendiente; eso no significa que una oficina real la haya recibido o vaya a atenderla.

Diapositiva 3 TypeScript define los datos | 1:35 a 2:40

[Abre solicitud.model.ts y señala Estudiante, NuevaSolicitud y Solicitud. Regresa a la presentación.]

La primera actividad está en el modelo de datos. Estudiante contiene nombre, código y correo. NuevaSolicitud representa los datos que llegan del formulario. Solicitud amplía esa estructura con un identificador, un estado y una fecha de creación.

Los estados usan un tipo con valores permitidos: pendiente, en revisión y resuelta. Si se intenta asignar otro texto desde el código, TypeScript puede advertir el error antes de ejecutar. Además, la configuración strict exige un manejo más cuidadoso de los tipos.

También se utilizan contenidos de JavaScript moderno: const para referencias que no se reasignan, let para contadores que cambian, funciones flecha, desestructuración y spread. El proyecto comparte código mediante import y export. Por esa organización en módulos no necesita un namespace global.

Diapositiva 4 Angular organiza la aplicación | 2:40 a 3:40

[Muestra brevemente solicitudes.module.ts y el constructor de solicitudes.service.ts o del componente listado.]

La segunda actividad es la estructura de Angular 16. Cada componente tiene una responsabilidad visible: listar solicitudes, registrar una nueva, mostrar un detalle o consultar el directorio. El módulo de solicitudes reúne las pantallas de ese proceso y se carga al entrar a sus rutas.

La lógica de los registros está en SolicitudesService. Los componentes reciben ese servicio mediante inyección de dependencias, en lugar de crear una instancia por su cuenta.

En las plantillas, la interpolación muestra valores; los eventos responden a acciones; y los enlaces de propiedades controlan elementos de la interfaz. Se utilizan asterisco ngFor para recorrer listas y asterisco ngIf para mostrar contenido según una condición. La búsqueda usa ngModel, mientras que el registro utiliza un formulario reactivo.

Diapositiva 5 Validar antes de registrar | 3:40 a 5:05

[Cambia al navegador. En Nueva solicitud, pulsa Registrar solicitud con los campos vacíos. Luego completa los datos de ejemplo del final del guion. Reserva unos 30 segundos para escribir.]

Ahora voy a mostrar el formulario. Primero intento registrar sin completar los campos. Aparecen mensajes que indican qué información falta y el registro no continúa. Esto evita guardar solicitudes incompletas.

El formulario está definido con FormBuilder. El nombre debe tener al menos tres caracteres útiles, el código debe empezar con A mayúscula y continuar con ocho dígitos, y el correo debe tener un formato válido. El asunto requiere al menos cinco caracteres y la descripción, veinte. Escribir únicamente espacios tampoco cumple esas condiciones.

Voy a completar una solicitud de constancia de estudios con datos ficticios. Antes de guardar, el método marca los controles como revisados y comprueba la validez del formulario. Si hay un error, se detiene antes de llamar al servicio. La validación ayuda al usuario a corregir el dato concreto y mantiene una estructura consistente en los registros.

Diapositiva 6 Registro y navegación | 5:05 a 6:05

[Guarda la solicitud válida. Muestra el detalle, vuelve al listado y recarga la página.]

Al guardar una solicitud válida, la aplicación genera su identificador, le asigna el estado pendiente y abre la pantalla de detalle. RouterModule conecta estas pantallas: el listado, la ruta nueva y la ruta con el identificador de la solicitud. La ruta nueva se declara antes que la dinámica para que no se interprete como un identificador.

Ahora regreso al listado y recargo la página. El registro permanece porque se guarda en localStorage, dentro del navegador. El servicio intenta guardar antes de comunicar el cambio a la vista. Si el almacenamiento falla, muestra el problema y conserva los datos del formulario.

Esta persistencia permite demostrar el flujo sin un servidor propio. Sin embargo, pertenece a este navegador y a este sitio: otro navegador tendrá su propia lista y borrar los datos del sitio eliminará los registros locales.

Diapositiva 7 Una consulta REST real | 6:05 a 7:25

[Abre Directorio y DevTools, pestaña Network. Actualiza, filtra users y muestra la URL, el método GET, el estado y la respuesta. Reserva unos 20 segundos para esta demostración.]

La cuarta actividad se demuestra en Directorio. La API utilizada es JSONPlaceholder, específicamente el recurso users. Es un servicio de prueba que devuelve usuarios ficticios. Estos contactos se muestran por separado y no son las solicitudes del formulario.

En la pestaña Network se puede comprobar la petición GET, la dirección consultada y la respuesta recibida. En la ejecución documentada, la API respondió con estado doscientos y diez usuarios.

DirectorioService recibe HttpClient por inyección de dependencias. Solicita la respuesta como unknown y revisa su estructura antes de usarla. Esto es importante porque una interfaz de TypeScript no valida automáticamente un JSON recibido durante la ejecución.

HttpClient devuelve un Observable. En el componente, firstValueFrom permite esperar su primera respuesta con async y await. El bloque try, catch y finally organiza la carga, el error y la finalización de la consulta.

Diapositiva 8 Errores y decisiones técnicas | 7:25 a 8:15

[Regresa a la presentación. La captura corresponde a una prueba con error HTTP 503 simulado, no a una caída comprobada de la API real.]

También se considera qué ocurre cuando la consulta falla. La aplicación muestra un mensaje y permite volver a intentarlo. La captura corresponde a un error quinientos tres simulado durante las pruebas; sirve para comprobar esa respuesta de la interfaz. Además, la consulta tiene un tiempo máximo de espera de doce segundos.

Se mantiene Angular 16 porque es la versión requerida para el caso. Angular CLI administra la compilación con Webpack, y los source maps ayudan a depurar el código original.

No se implementa un backend Node.js porque no es requisito técnico de esta PA1. En una etapa posterior, un servidor podría recibir las solicitudes mediante endpoints, validar los datos y guardarlos en una base central. Aquí el alcance es el frontend y el consumo de una API disponible.

Diapositiva 9 Evidencias de funcionamiento | 8:15 a 9:10

[Abre evidencias/pruebas.txt y muestra la línea 18 passed. Si queda tiempo, muestra la captura móvil.]

Para revisar el resultado, se dispone de evidencias de tipado, compilación y pruebas de navegador. La ejecución guardada registra dieciocho pruebas aprobadas. Se revisan casos como el formulario inválido, el registro correcto, la persistencia, los filtros y el manejo de errores.

Un caso importante es el almacenamiento bloqueado: la aplicación no debe anunciar un registro exitoso si no pudo guardarlo. Otro es la consulta externa: se diferencia la prueba contra la API real de las pruebas que simulan respuestas para controlar errores.

Las capturas también muestran la versión móvil. El README explica el objetivo, la estructura y las instrucciones de ejecución. Para iniciar el proyecto se instalan las dependencias con npm ci y se ejecuta npm start, usando una versión de Node compatible con lo indicado en el repositorio.

Diapositiva 10 Conclusiones | 9:10 a 10:00

[Vuelve a la última diapositiva y cierra mirando a la cámara. No afirmes que el repositorio o el video están publicados si aún falta hacerlo.]

Para concluir, el proyecto permite seguir una solicitud desde su ingreso hasta su consulta y relacionar ese recorrido con los contenidos de clase. TypeScript define la estructura de los datos; Angular separa las pantallas y los servicios; el formulario reactivo aplica las reglas; y HttpClient permite consultar una fuente externa.

Una diferencia importante es que tipar los datos y validarlos son tareas complementarias. También se distingue entre guardar información localmente y enviarla a un servidor: las solicitudes de Campus permanecen en el navegador, mientras el directorio consulta JSONPlaceholder.

El siguiente paso funcional sería incorporar un backend y usuarios autenticados para compartir registros y gestionar cambios de estado reales. Para esta PA1, el resultado demuestra las cuatro actividades dentro del alcance solicitado. Con esto termina la presentación. Muchas gracias, profesor.

Preparación para grabar

Esta página es una ayuda para ensayar y preparar la grabación. No forma parte de los diez minutos de explicación.

Antes de empezar

Mantén la cámara encendida durante toda la exposición. Abre el PowerPoint, la aplicación y el editor con los archivos que vas a mostrar. Comprueba Internet, micrófono y tamaño del texto. Ensaya con cronómetro; ajusta las pausas a tu ritmo sin omitir la demostración.

Datos ficticios para el formulario

Nombre: Andrea Ejemplo

Código: A20261234

Correo: andrea@example.com

Tipo: Constancia de estudios

Asunto: Constancia para prácticas profesionales

Descripción: Solicito una constancia de estudios para adjuntar a mi postulación de prácticas profesionales.

Archivos para mostrar

En src/app/core/models: solicitud.model.ts. En src/app/features/solicitudes: solicitudes.module.ts y lista-solicitudes.component.ts. En evidencias: pruebas.txt. Puedes abrirlos antes de grabar para evitar búsquedas durante la exposición.

Antes de entregar

Confirma con el docente la modalidad individual, porque la consigna pide trabajo en equipo. Describe únicamente aportes que realmente puedas explicar. Verifica que el repositorio contenga el proyecto y sea accesible para revisión. Publica tu video en YouTube con visibilidad Público y coloca su enlace real en el README. La asistencia y participación requieren evidencias reales adicionales.